- **自执行函数**
  - 自执行函数是什么？
    - 是一种将函数声明和调用合并到一起的写法，我们称为自执行函数或立即执行函数

```
// 将普通函数用小括号 () 包裹，然后再用 () 直接调用
;(function bar() {
  console.log('函数立即被执行了...');
})()

// 自执行函数一般都采用匿名函数
;(function () {
  console.log('自执行函数...');
})()
```

注意：**自执行函数前面尽量都加上分号**

1. 不然会引起错误
2. 后期我们压缩代码也会引起错误
3. 常见的除了分号之外也有 ！ 或者+
4. 但是加分号是我们最常见的写法

    - 自执行函数两种常见写法

```
// 第一种方式 常用  (function(){})()
;(function () {
    console.log('第一种方式')
})()
// 第二种方式   (function(){}())
;(function () {
   console.log('第二种方式')
}())
```

    - 自执行函数也可以传递参数
      - 自执行函数也普通函数并无本质的区别，同样具有形参与实参

```
//; (function (此处形参) {
//  // 此处函数体
// })(此处实参)

;(function (name, age) {
  console.log(`大家好，我叫${name}，我今年${age}岁了。`);
})('小明', 18)
```

- **命名空间**
  - 语法：

```
// pinkDate 就是一个命名空间
let pinkDate = {
    author: 'pink德华',
    date: new Date(),
    getYear: function () {
        return this.date.getFullYear()
    },
    getMonth: function () {
        return this.date.getMonth() + 1
    }
}
// 使用命名空间
document.write(`今年是：${pinkDate.getYear()}年${pinkDate.getMonth()}月`)
```

- **行内事件**
  - 什么是行内事件?
    - 监听一个元素的事件，也可以写在标签内，这种写法称之为行内事件
  - 语法：

```
<标签 on事件名="代码"></标签>
```

举例说明

```
<button onclick="alert('我被点了')">按钮</button>
```

引号经常采用外双内单

  - 行内事件也可以调用函数

```
<button onclick="foo()">按钮</button>
<script>
    function foo() {
        // 因为函数内部可以写很多代码
        console.log('可以调用函数')
    }
</script>
```

```
<button onclick="foo(1,2)">按钮</button>
<script>
    function foo(x, y) {
        // 因为函数内部可以写很多代码
        console.log(x + y)
    }
</script>
```

  - 如果事件触发时要执行多句代码，一般会调用函数
    - 此时也可以给函数传参
    - 注：此时函数内的this是window
    - 在后面的Vue中，会有类似的用法，现在作为了解

- **L0事件**
  - DOM L0 指的是 DOM 刚诞生时的版本
  - 语法：

```
元素.on事件名 = 触发事件时的函数
```

举例说明

```
btn.onclick = function () {
    alert('我被点了')
}
```

还是有事件三要素：事件源、事件类型、处理程序

- 这种写法就是在事件类型前固定加on，即监听此类事件
- 我们之前学的 addEventListener 是 L2 事件（推荐使用）

- DOM L0 和 DOM L2 区别
  - DOM L0 写法：相同事件类型只能绑定一次
  - DOM L2 写法：相同事件类型可以绑定多次

- **事件移除**
  - L0 事件移除：

```
元素.on事件名 = null
```

举例说明

```
btn.onclick = null
```

  - L2 事件移除：

```
元素.removeEventListener('事件类型', 要移除的回调函数)
```

举例说明

```
btn.removeEventListener('click', handle)
```

注：如果添加事件监听时是用的匿名函数，则无法移除

- **事件类型**
  - 鼠标事件
    - 是指跟鼠标操作相关的事件，如单击、双击、移动等。
  - 常见的鼠标事件:

事件类型	作用	说明
click	鼠标单击	鼠标点击一次
dblclick	鼠标双击	快速双击两次
mouseover /mouseenter	鼠标移入	鼠标经过元素会触发
mouseout /mouseleave	鼠标移出	鼠标离开元素会触发
mousemove	鼠标移动	鼠标在元素上移动会触发
mousedown	鼠标按下	鼠标按下的时候触发
mouseup	鼠标弹起	鼠标弹起的时候触发

- **事件对象**
  - 事件对象是什么
    - 也是个对象，这个对象里有事件触发时的相关信息
    - 例如：鼠标点击事件中，事件对象就存了鼠标点在哪个位置等信息
  - 如何获取
    - 在事件绑定的回调函数的第一个参数就是事件对象
    - 一般命名为event、ev、e
  - 部分常用属性
    - type
      - 获取当前的事件类型
    - clientX/clientY
      - 获取光标相对于浏览器可见窗口左上角的位置
    - pageX/pageY
      - 获取光标相对于整个文档左上角的位置
    - offsetX/offsetY
      - 获取光标相对于当前DOM元素左上角的位置